



厦门大学
XIAMEN UNIVERSITY

Python函数



01 函数

1.1 函数及其功能

- 函数是一段具有特定功能的、可重用的语句组，用函数名来表示并通过函数名完成功能调用
- 函数可看作是一段具有名字的子程序，可在需要的地方调用执行，不需要在每个执行地方重复编写这些语句
- 每次使用函数可以提供不同的参数作为输入，以实现对不同数据的处理
- 函数执行后，还可以反馈相应的处理结果。
- 函数能提高应用的模块性，和代码的重复利用率

1.2 函数定义

- Python定义函数的格式如下：

def 函数名 (<参数表>):

函数体

- 函数代码块以def关键词开头，后接函数标识符名称和圆括号()。
- 传入参数和自变量必须放在圆括号中间。圆括号之间可用于定义参数。
- 函数的第一行语句可以选择性地使用文档字符串—用于存放函数说明。
- 函数内容以冒号起始，并且缩进。
- return[表达式]结束函数，返回一个值给调用方。
- 不带表达式的return相当于返回None

函数



```
print("Happy birthday to you!")
print("Happy birthday toyou!")
print("Happy  birthday, dear
Mike!")
print("Happy birthday to you!")
def happy():
    print("Happy birthday to you!")
def happyB(name):
    happy()
    happy()
    print("Happy
birthday, dear {}".format(name))
```

```
print("下面展示函数调用情况:")
happy()
happyB("Mike")
```

下面展示函数调用情况:

```
Happy birthday to you!
Happy birthday to you!
Happy birthday to you!
Happy birthday, dear
Mike!
```

1.3 函数调用

实际上之前我们都调用过函数，例如：Number=int(String, 10)

调用函数的时候，赋值符号左边的变量用来接收函数的返回值

有几个返回值就要有几个变量

没有返回值的函数可直接调用，例如强制退出函数exit(0)

```
def demo_function(a_number, a_string, a_list):  
    a_number = pow(a_number, 3)      #pow: 返回x的y次方的值  
    print(a_string.lower())  
    print(a_string.upper())  
    map(abs, a_list)  
    return a_number, a_list  
new_number, new_list = demo_function(Number, String, List)
```

1.3 函数的调用

程序调用一个函数需要执行以下四个步骤：

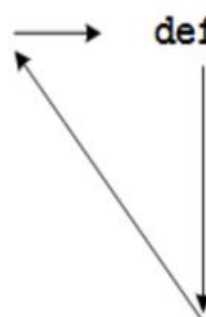
- (1) 调用程序在调用处暂停执行；
- (2) 在调用时将实参复制给函数的形参；
- (3) 执行函数体语句；
- (4) 函数调用结束给出返回值，程序回到调用前的暂停处继续执行

`name="Mike"`

```
happyB("Mike")  
print()  
happyB("Lily")
```

→

```
def happyB(name):  
    happy()  
    happy()  
    print("Happy birthday, dear!".format(name))  
    happy()
```



1.4.1 函数的参数传递

函数定义中可能包含多个形参，因此函数调用中也可能包含多个实参。

向函数传递实参的方式很多：

- 可使用位置实参，这要求实参的顺序与形参的顺序相同；
- 可使用关键字实参，其中每个实参都由变量名和值组成；
- 可使用列表和字典。

1.4.1 函数的参数传递

➤ 位置实参

Python将函数调用中的每个实参都关联到函数定义中的一个形参，最简单的关联方式是基于实参的顺序，这种关联方式被称为位置实参。

函数的调用中，实参存储在形参中，位置实参的顺序很重要，实参的顺序不正确的话，可能会有不可预知问题；

➤ 关键字实参

关键字实参是传递给函数的名称—值对。在实参中将名称和值关联起来了，因此向函数传递实参时不会混淆（不会得到名为Hamster的harry这样的结果）。

关键字实参让你无需考虑函数调用中的实参顺序，还清楚地指出了函数调用中各个值的用途。

1.4.2 函数的参数传递实例

位置参数：调用函数时必须传入的参数，少或者多均会报错

```
def demo_function(a_number, a_string, a_list):  
    a_number = pow(a_number, 3)  
    print(a_string.lower())  
    print(a_string.upper())  
    map(abs, a_list)  
    return a_number, a_list  
  
new_number, new_list = demo_function(Number, String, List)
```

三个参数都是位置参数，如果调用函数时传入的参数个数不是三个就会报错

函数的参数传递：默认值

- 编写函数时，可给每个形参指定默认值。
- 在调用函数中给形参提供了实参时，Python将使用指定的实参值；否则，将使用形参的默认值。
- 给形参指定默认值后，可在函数调用中省略相应的实参。
- 使用默认值可简化函数调用，还可清楚地指出函数的典型用法。
- 如果显式地给`animal_type`提供了实参，因此Python将忽略这个形参的默认值。使用默认值时，在形参列表中必须先列出没有默认值的形参，再列出有默认值的形参。这让Python依然能够正确地解读位置实参。

避免实参错误

使用函数时，如果遇到实参不匹配错误，不要大惊小怪。你提供的实参多于或少于函数完成其工作所需的信息时，将出现实参不匹配错误。

函数的参数传递实例

默认参数：在定义函数时赋予其一个默认值，如果调用函数时在该位置没有传入参数则使用默认值。

```
def demo_function(a_number, a_string, a_list = [1, -2, 3]):  
    a_number = pow(a_number, 3)  
    print(a_string.lower())  
    print(a_string.upper())  
    a_list = list(map(abs, a_list))  
    return a_number, a_list
```

参数表中有2个位置参数，1个默认参数

可变参数：传入的任意个数的实参

例如：定义一个sum函数，对函数传入的若干个数进行求和。

```
Def    sum(*number):  
    s=0  
    for n in number:  
        s+=n  
    return s
```

实际上，Python会自动把传入*number（可变参数）的参数放入一个tuple（元组）内，所以number实际上就是一个tuple（元组）。将组合数据类型作为可变参数传入函数时，只需要在数据名称前面加入*即可，例如
sum(*num_list)

调用函数时，可以指定参数名（关键字）从而可以不考虑参数的传入顺序。例如：

```
def demo_function(a_number, a_string, a_list = [1, -2, 3]):  
    a_number = pow(a_number, 3)  
    print(a_string.lower())  
    print(a_string.upper())  
    a_list = list(map(abs, a_list))  
    return a_number, a_list
```

```
new_number, new_list = demo_function(Number, a_list = List, a_string = String)
```

4、关键字参数：当需要传入的参数个数未知，且每个参数都有名字时，可以使用关键字参数。

例如：我们定义一个创建用户信息的函数，其中姓名，性别与年龄是必选参数，但是有时会传入地区、职业、文化程度等其他信息，这些信息可以是0个或多个，可这样定义函数：

```
def create_a_person(name, sex, age, **kw)
```

Python会自动把传入**kw（关键字参数）的参数放入一个dict（字典）内，kw实际上是一个dict（字典），其中键是参数名，值是参数值。

函数的参数传递



厦门大学
XIAMEN UNIVERSITY

```
def create_a_person(name, sex, age, **kw):  
    print("This person's name is {}".format(name))  
    print("This person's sex is {}".format(sex))  
    print("This person's age is {}".format(age))  
  
    for key, value in kw.items():  
        print("This person's {} is {}".format(key, value))  
  
    print()
```

```
create_a_person("Alice", "female", 20, area = "Xiamen", job = "Student")  
create_a_person("Bob", "male", 30, area = "Beijing", job = "Teacher", education = "PHD")
```

This person's name is Alice
This person's sex is female
This person's age is 20
This person's area is Xiamen
This person's job is Student

This person's name is Bob
This person's sex is male
This person's age is 30
This person's area is Beijing
This person's job is Teacher
This person's education is PHD

进程已结束,退出代码0

5、结合使用位置实参和任意数量实参

有时需要指定关键字参数的名字，例如在上一个示例中，我们要求必须指定用户所在的地区信息，这时可以综合用到关键字参数和任意数量实参，定义函数如下：

```
def create_a_person(name, sex, age, *, city, **kw):
```

即在命名关键字之前加一个*,
可以给命名关键字参数定义
默认值

```
def create_a_person(name, sex, age, *, area = None, **kw):  
    print("This person's name is {}".format(name))  
    print("This person's sex is {}".format(sex))  
    print("This person's age is {}".format(age))  
  
    if area != None:  
        print("This person's area = {}".format(area))  
  
    for key, value in kw.items():  
        print("This person's {} is {}".format(key, value))
```

5、命名关键字参数：

注意：

- ①当参数表有可变参数时，命名关键字参数之前不需要加*，
- ②命名关键字参数与位置参数的区别是：调用函数时，可以不指定位置参数的参数名，但一定要指定命名关键字参数的参数名（如果需要传入），否则会报错。

定义函数时，参数表中参数类型的出现顺序：

位置参数、默认参数、可变参数、命名关键字参数、关键字参数

```
def create_a_person(name, sex, age, *assets, area, job = None, **kw):
    print("This person's name is {}".format(name))
    print("This person's sex is {}".format(sex))
    print("This person's age is {}".format(age))
    print("This person's area = {}".format(area))

    all_assets = 0
    for i in assets:
        all_assets += i
    print("This person's assets is {}".format(all_assets))

    if job != None:
        print("This person's job = {}".format(job))

    for key, value in kw.items():
        print("This person's {} is {}".format(key, value))

print()

Alice_assets = [100, 3000, 817, 9999]
Bob_assets = [73291873, 328173, 89721893, 3123897, 656134]
create_a_person("Alice", "female", 20, *Alice_assets, area = "Xiamen", hobby = "sing and dance")
create_a_person("Bob", "male", 30, *Bob_assets, area = "Beijing", job = "Teacher", education = "PHD")
```

```
This person's name is Alice
This person's sex is female
This person's age is 20
This person's area = Xiamen
This person's assets is 13916
This person's hobby is sing and dance
```

```
This person's name is Bob
This person's sex is male
This person's age is 30
This person's area = Beijing
This person's assets is 167121970
This person's job = Teacher
This person's education is PHD
```

进程已结束,退出代码0

- 函数并非总是直接显示输出，相反，它可以处理一些数据，并返回一个或一组值。
- 在函数中，可使用return语句将值返回到调用函数的代码行。
- 返回值让你能够将程序的大部分繁重工作移到函数中去完成，从而简化主程序。

```
def sum(arg1, arg2):  
    #返回2个参数的和."  
    total=arg1+arg2  
    print"函数内  
:", total  
    return total
```

```
#调用sum函数  
total=sum(10, 20)
```

- 函数体中return语句有指定返回值时返回的就是其值
- 返回字典函数可返回任何类型的值，包括列表和字典等较复杂的数据结构
- 函数体中没有return语句时，函数运行结束会隐含返回一个None作为返回值，类型是NoneType，与return、returnNone等效，都是返回None。

- 一个程序中的变量包括两类：全局变量和局部变量。
- 全局变量指在函数之外定义的变量，一般没有缩进，在程序执行全过程有效。
- 局部变量指在函数内部使用的变量，在函数内部有效，当函数退出时变量不存在。

```
n = 1 #n是全局变量
def func(a, b):
    c = a * b #c是局部变量, a和b作为函数参数也是局部变量
    return c
s = func("knock~", 2)
print(c)
```

```
-----
NameError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_25480\3084982553.py in <module>
      4     return c
      5 s = func("knock~", 2)
----> 6 print(c)

NameError: name 'c' is not defined
```

如果希望让func()函数将n当作全局变量，需要在变量n使用前显式声明该变量为全局变量

```
n = 1 #n是全局变量
def func(a, b):
    n = b #这个n是在函数内存中新生成的局部变量
    return a*b
s = func("knock~", 2)
print(s, n) #测试一下n值是否改变
```

knock~knock~ 1

```
n = 1 #n是全局变量
def func(a, b):
    global n
    n = b #将局部变量b赋值给全局变量n
    return a*b
s = func("knock~", 2)
print(s, n) #测试一下n值是否改变
```

knock~knock~ 2

函数对变量的作用



此时的全局变量不是整数n，而是列表类型ls，则会直接修改列表

```
ls = [] #ls是全局列表变量
def func(a, b):
    ls.append(b) #将局部变量b增加到全局列表变量ls中
    return a*b
s = func("knock~", 2)
print(s, ls) #测试一下ls值是否改变
```

knock~knock~ [2]

如果函数内部存在一个真实创建过且名称为ls的列表，则函数将操作该列表而不会修改全局变量

```
ls = [] #ls是全局列表变量
def func(a, b):
    ls = [] #创建了名称为ls的局部列表变量
    ls.append(b) #将局部变量b增加到全局列表变量ls中
    return a*b
s = func("knock~", 3)
print(s, ls) #测试一下ls值是否改变
```

knock~knock~knock~ []

Python函数对变量的作用遵守如下原则：

- 简单数据类型变量无论是否与全局变量重名，仅在函数内部创建和使用，函数退出后变量被释放；
- 简单数据类型变量在用global保留字声明后，作为全局变量；
- 对于组合数据类型的全局变量，如果在函数内部没有被真实创建的同名变量，则函数内部可直接使用并修改全局变量的值；
- 如果函数内部真实创建了组合数据类型变量，无论是否有同名全局变量，函数仅对局部变量进行操作。

- 函数是程序的一种基本抽象方式，它将一系列代码组织起来通过命名供其他程序使用。
- 函数封装的直接好处是代码复用，任何其他代码只要输入参数即可调用函数，从而避免相
- 同功能代码在被调用处重复编写。
- 代码复用产生了另一个好处，当更新函数功能时，所有被调用处的功能都被更新
- 当程序的长度在百行以上，如果不划分模块就算是最好的程序员也很难理解程序含义程序的可读性就已经很糟糕了。
- 解决这一问题的最好方法是将一个程序分割成短小的程序段，每一段程序完成一个小的功能。
- 无论面向过程和面向对象编程，对程序合理划分功能模块并基于模块设计程序是一种常用方法，被称为“模块化设计”

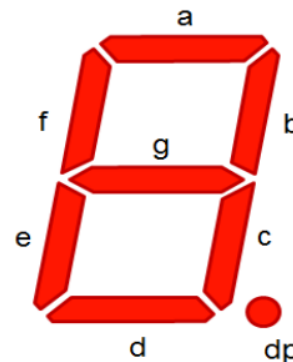
模块化设计一般有两个基本要求：

- 紧耦合：尽可能合理划分功能块，功能块内部耦合紧密；
 - 松耦合：模块间关系尽可能简单，功能块之间耦合度低。
-
- 使用函数只是模块化设计的必要非充分条件，根据计算需求合理划分函数十分重要。
 - 完成特定功能或被经常复用的一组语句应该采用函数来封装，并尽可能减少函数间参数
 - 和返回值的数量

七段数码管绘制



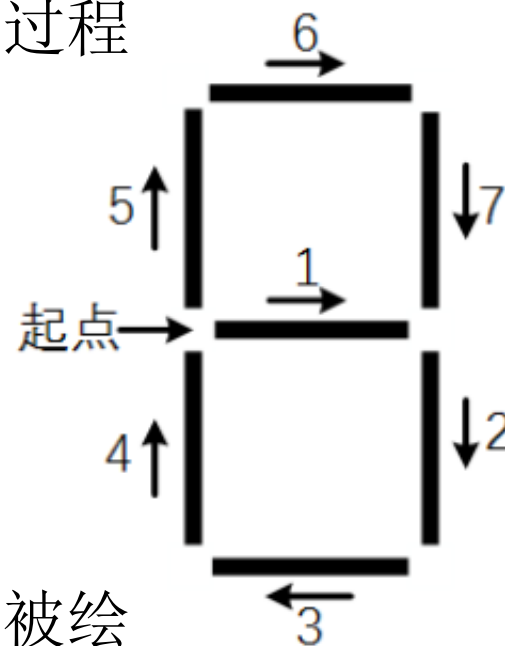
七段数码管（seven-segment indicator）由7段数码管拼接而成，每段有亮或不亮两种情况，改进型的七段数码管还包括一个小数点位置，如图所示



七段数码管绘制



- 每个0到9的数字都有相同的七段数码管样式，可通过设计函数复用数字的绘制过程。
- 每个七段数码管包括7个数码管样式，除了数码管位置不同外，绘制风格一致，也可以通过函数复用单个数码段的绘制过程
- 实例代码定义了drawDigit()函数，
- 该函数根据输入的数字d绘制七段数码管，
- 结合七段数码管结构，每个数码管的绘制采用图所示顺序。
- 绘制起点在数码管中部左侧，无论每段数码管是否被绘制出来，turtle画笔都按顺序“画完”所有7个数码管。
- 对于给定数字d，哪个数码段被绘制出来采用if...else...语句判断



七段数码管绘制



厦门大学
XIAMEN UNIVERSITY

```
import turtle,datetime
def drawLine(draw):#绘制单段数码管
    turtle.pendown() if draw else turtle.penup()
    turtle.fd(40)
    turtle.right(90)
def drawDigit(digit):#根据数字绘制七段数码管
    drawLine(True) if digit in[2,3,4,5,6,8,9] else drawLine(False)
    drawLine(True) if digit in[0,1,3,4,5,6,7,8,9] else drawLine(False)
    drawLine(True) if digit in[0,2,3,5,6,8,9] else drawLine(False)
    drawLine(True) if digit in[0,2,6,8] else drawLine(False)
    turtle.left(90)
    drawLine(True) if digit in[0,4,5,6,8,9] else drawLine(False)
    drawLine(True) if digit in[0,2,3,5,6,7,8,9] else
    drawLine(False)
    drawLine(True) if digit in[0,1,2,3,4,7,8,9] else drawLine(False)
    turtle.left(180)
    turtle.penup()
    turtle.fd(20)
def drawDate(date):#获得要输出的数字
    for i in date:
        drawDigit(eval(i))#注意:通过eval()函数将数字变为整数
def main():
    turtle.setup(800,350,200,200)
    turtle.penup()
    turtle.fd(-300)
    turtle.pensize(5)
    drawDate(datetime.datetime.now().strftime('%Y%m%d'))
    turtle.hideturtle()
main()
```

202203 14

七段数码管绘制



厦门大学
XIAMEN UNIVERSITY

```
import turtle,datetime
def drawGap():#绘制数码管间隔
    turtle.penup() #提起笔移动，不绘制图形，用于另起一个地方绘制
    turtle.fd(5)
def drawLine(draw):#绘制单段数码管
    drawGap()
    turtle.pendown() if draw else turtle.penup()
    turtle.fd(40)
    drawGap()
    turtle.right(90)
def drawDigit(d):#根据数字绘制七段数码管
    drawLine(True) if d in[2,3,4,5,6,8,9] else drawLine(False)
    drawLine(True) if d in[0,1,3,4,5,6,7,8,9] else
drawLine(False)
    drawLine(True) if d in[0,2,3,5,6,8,9] else drawLine(False)
    drawLine(True) if d in[0,2,6,8] else drawLine(False)
    turtle.left(90)
    drawLine(True) if d in[0,4,5,6,8,9] else drawLine(False)
    drawLine(True) if d in[0,2,3,5,6,7,8,9] else drawLine(False)
    drawLine(True) if d in[0,1,2,3,4,7,8,9] else drawLine(False)
    turtle.left(180)
    turtle.penup()
    turtle.fd(20)
```

2022 年 03 月 14 日

```
def drawDate(date):
    turtle.pencolor("red")
    for i in date:
        if i == '-':
            turtle.write('年',font=("Arial",18,"normal"))
            turtle.pencolor("green")
            turtle.fd(40)

        elif i == '=':
            turtle.write('月',font=("Arial",18,"normal"))
            turtle.pencolor("blue")
            turtle.fd(40)
        elif i == '+':
            turtle.write('日',font=("Arial",18,"normal"))
        else:
            drawDigit(eval(i))
def main():
    turtle.setup(800,350,200,200) #width, height, startx, starty
    turtle.penup()
    turtle.fd(-350)
    turtle.pensize(5)
    drawDate(datetime.datetime.now().strftime('%Y-%m=%d+'))
    turtle.hideturtle()
main()
```

在函数内部可以调用别的函数，也可以调用函数本身，此时称为函数的递归。递归一般需要有递归出口，否则将陷入无限递归。

阶乘，阶乘通常定义：

$$n! = n(n-1)(n-2)\dots(1)$$

进一步的，可以将上式转换为：

$$n! = \begin{cases} 1 & n = 0 \\ n * (n-1)! & \text{otherwise} \end{cases}$$

阶乘的例子揭示了递归的2个关键特征：

- (1) 存在一个或多个基例，基例不需要再次递归，它是确定的表达式；
- (2) 所有递归链要以一个或多个基例结尾

```
def f(n):  
    if n==1:  
        return n  
    else:  
        return n*f(n-1)  
result = f(10) #计算10的阶乘  
print(result)
```

3628800

科赫曲线的基本概念和绘制方法如下：

- 正整数 n 代表科赫曲线的阶数，表示生成科赫曲线过程的操作次数。
- 科赫曲线初始化阶数为0，表示一个长度为 L 的直线。
- 对于直线 L ，将其等分为三段，中间一段用边长为 $L/3$ 的等边三角形的两个边替代，得到1阶科赫曲线，它包含四条线段。
- 进一步对每条线段重复同样的操作后得到2阶科赫曲线。继续重复同样的操作 n 次可以得到 n 阶科赫曲线

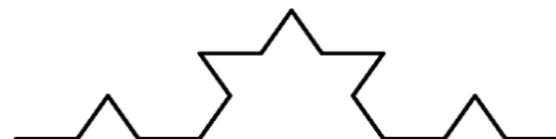
0阶科赫曲线



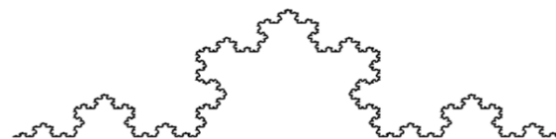
1阶科赫曲线



2阶科赫曲线



5阶科赫曲线

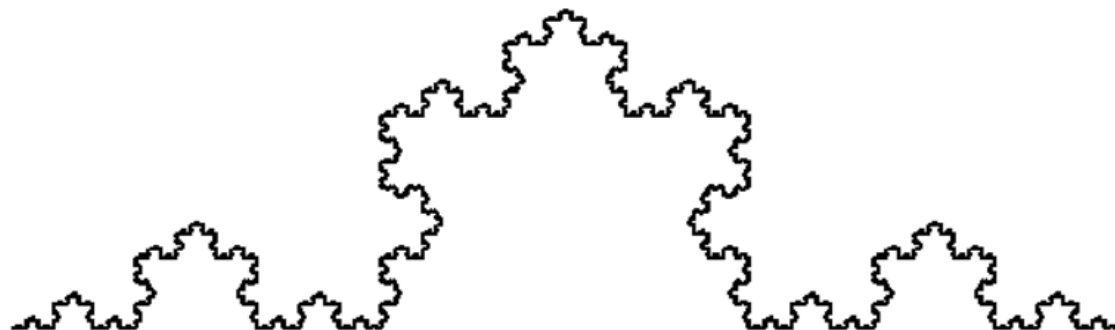


科赫曲线绘制



厦门大学
XIAMEN UNIVERSITY

```
import turtle
def koch(size,n):
    if n==0:
        turtle.fd(size)
    else:
        for angle in[0,60,-120,60]:
            turtle.left(angle)
            koch(size/3,n-1)
def main():
    turtle.setup(800,400)
    turtle.speed(0)#控制绘制速度
    turtle.penup()
    turtle.goto(-300,-50)
    turtle.pendown()
    turtle.pensize(2)
    koch(600,6)#0阶科赫曲线长度, 阶数
    turtle.hideturtle()
main()
```



- 应给函数指定描述性名称，且只在其中使用小写字母和下划线。描述性名称可帮助你和别人明白代码想要做什么。
- 给模块命名时也应遵循上述约定。每个函数都应包含简要地阐述其功能的注释，该注释应紧跟在函数定义后面，并采用文档字符串格式。
- 文档良好的函数让其他程序员只需阅读文档字符串中的描述就能够使用它：代码如描述的那样运行；只要知道函数的名称、需要的实参以及返回值的类型，就能在自己的程序中使用它。
- 给形参指定默认值时，等号两边不要有空格：
`def function_name(parameter_0, parameter_1='defaultvalue')`
- 对于函数调用中的关键字实参，也应遵循这种约定：
`function_name(value_0, parameter_1='value')`
- PEP8 (<https://www.python.org/dev/peps/pep-0008/>) (Python编码规范) 建议代码行的长度不要超过79字符，这样只要编辑器窗口适中，就能看到整行代码。

- 如果形参很多，导致函数定义的长度超过了79字符，可在函数定义中输入左括号后按回车键，并在下一行按两次Tab键，从而将形参列表和只缩进一层的函数体区分开来。
- 大多数编辑器都会自动对齐后续参数列表行，使其缩进程度与你给第一个参数列表行指定的缩进程度相同：

```
def function_name(parameter_0, parameter_1, parameter_2, parameter_3, parameter_4, parameter_5):  
    functionbody...
```

- 如果程序或模块包含多个函数，可使用两个空行将相邻的函数分开，这样将更容易知道前一个函数在什么地方结束，下一个函数从什么地方开始。所有的import语句都应放在文件开头，唯一例外的情形是，在文件开头使用了注释来描述整个程序。

调用别的库的函数的方法



厦门大学
XIAMEN UNIVERSITY

例如`sqrt()`，`sin()`，`cos()`等函数在`math`库。

首先要在调用该函数的语句之前（一般放在代码开头）导入该包：

```
import math
```

调用时：`math.sqrt(9)`，`math.sin(x)`，`math.cos(0)`等

即库名+点号+函数

Python内置函数



厦门大学
XIAMEN UNIVERSITY

<code>abs()</code>	<code>id()</code>	<code>round()</code>	<code>compile()</code>	<code>locals()</code>
<code>all()</code>	<code>input()</code>	<code>set()</code>	<code>dir()</code>	<code>map()</code>
<code>any()</code>	<code>int()</code>	<code>sorted()</code>	<code>exec()</code>	<code>memoryview()</code>
<code>ascii()</code>	<code>len()</code>	<code>str()</code>	<code>enumerate()</code>	<code>next()</code>
<code>bin()</code>	<code>list()</code>	<code>tuple()</code>	<code>filter()</code>	<code>object()</code>
<code>bool()</code>	<code>max()</code>	<code>type()</code>	<code>format()</code>	<code>property()</code>
<code>chr()</code>	<code>min()</code>	<code>zip()</code>	<code>frozenset()</code>	<code>repr()</code>
<code>complex()</code>	<code>oct()</code>		<code>getattr()</code>	<code>setattr()</code>
<code>dict()</code>	<code>open()</code>		<code>globals()</code>	<code>slice()</code>
<code>divmod()</code>	<code>ord()</code>	<code>bytes()</code>	<code>hasattr()</code>	<code>staticmethod()</code>
<code>eval()</code>	<code>pow()</code>	<code>delattr()</code>	<code>help()</code>	<code>sum()</code>
<code>float()</code>	<code>print()</code>	<code>bytearray()</code>	<code>isinstance()</code>	<code>super()</code>
<code>hash()</code>	<code>range()</code>	<code>callable()</code>	<code>issubclass()</code>	<code>vars()</code>
<code>hex()</code>	<code>reversed()</code>	<code>classmethod()</code>	<code>iter()</code>	<code>__import__()</code>



02 高阶函数

在Python中，变量可以指向函数，函数可以作为另一个函数的参数，函数还可以作为结果返回，这些都是高阶函数。

1、变量指向函数：相当于给函数更名。

```
# 定义一个阶乘函数
def f(n):
    if n == 1:
        return n
    else:
        return n * f(n - 1)
```

```
result = f(10) # 10的阶乘
print("result = {}".format(result))

factor = f
result2 = factor(10)
print("result2 = {}".format(result2))
```

```
result = 3628800
result2 = 3628800
```

进程已结束,退出代码0

如果给函数名赋值，则函数名会变成变量，无法再调用

2、函数可以作为另一个函数的参数。

```
# 定义一个阶乘函数
def factor(n):
    if n == 1:
        return n
    else:
        return n * factor(n - 1)

result = 403200

进程已结束,退出代码0

def my_sum_fuction(x, y, f):
    return f(x) + f(y)

result = my_sum_fuction(8, 9, factor) # 计算8的阶乘与9的阶乘的和
print("result = {}".format(result))
```


3、函数可以作为结果返回。

```
def lazy_sum(*number_list):  
    def sum():  
        result = 0  
        for i in number_list:  
            result += i  
        return result  
    return sum  
  
result = lazy_sum(1, 3, 5, 7, 9)  
print(result)  
print(result())
```

```
<function lazy_sum.<locals>.sum at 0x000002403125B430>
```

25

当调用`lazy_sum()`时，返回的是`sum()`函数，所以相当于`result=sum`，当调用`result()`函数（即`sum()`函数）时，才会返回结果。内部函数`sum`可以引用外部函数`lazy_sum`的参数和局部变量，当`lazy_sum`返回函数`sum`时，相关参数和变量都保存在返回的函数中，这种结构称为“闭包（Closure）”。

1、map函数：

接收两个参数，一个是函数，一个是list。map()函数将传入的函数依次作用到每个元素，并把结果作为一个新的迭代器返回。

```
def f(x):  
    return x * x
```

```
[1, 4, 9, 16, 25]
```

```
List = [1, 2, 3, 4, 5]  
List2 = list(map(f, List))  
print(List2)
```

```
进程已结束,退出代码0
```

2、reduce函数（需要导入库functools）：

把函数作用在序列上进行累积，要求函数必须是二元的（即两个函数），
相当于返回 $f(f(f(\cdots f(x_1, x_2), x_3, \cdots))$

```
import functools as f
```

```
def add(x, y):  
    return x + y
```

15

进程已结束,退出代码0

```
List = [1, 2, 3, 4, 5]  
print(f.reduce(add, List))
```

3、filter函数:

起筛选作用，把函数作用在list上，若返回True则保留，否则删除之，最后返回一个迭代器。例如下面筛选1-100以内的所有素数

```
def is_prime(x):  
    if x < 2:  
        return False  
    else:  
        for i in range(2, int(pow(x, 0.5)) + 1):  
            if x % i == 0:  
                return False  
        return True
```

[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

进程已结束,退出代码0

```
List = []  
for i in range(1, 101):  
    List.append(i)
```

```
List2 = list(filter(is_prime, List))  
print(List2)
```

53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

4、sorted函数：

一共有3个参数，分别是iterable, key, reverse，其中iterable是可迭代对象（比如list, tuple, set），key函数可有可无，若有，将key函数作用在每个元素后再进行比较，reverse是升序或降序，True为降序，False为升序（默认），之后返回一个list

```
List = [-4, 3, 1, 0, -9] # 以绝对值降序对List进行排序  
List2 = sorted(List, key = abs, reverse = True)  
print(List2)
```

[-9, -4, 3, 1, 0]

进程已结束,退出代码0



03 匿名函数

匿名函数的定义方法：（需要用上保留字lambda）

lambda变量名:公式

特点：方便，不用另起函数名

```
print(list(map(lambda x: x * x, [1, 2, 3, 4, 5])))
```

```
[1, 4, 9, 16, 25]
```

进程已结束,退出代码0

```
def f(x):  
    return x * x
```

示例：一个List中每个元素都是一个二元有序对，代表每个人的姓名与分数，现在按分数降序对List进行排名。

```
List = [  
    ("Alice", 142), ("Bob", 128), ("Cindy", 120),  
    ("David", 108), ("Eric", 110), ("Frank", 129),  
    ("Gorge", 120), ("Hana", 131)  
]
```

```
print(sorted(List, key = lambda x: x[1], reverse = True))
```

```
[('Alice', 142), ('Hana', 131), ('Frank', 129), ('Bob', 128), ('Cindy', 120), ('Gorge', 120), ('Eric', 110), ('David', 108)]
```

进程已结束,退出代码0



04 偏函数

需要调用functools库，使用functools.partial函数。

作用：把一个函数的某些参数固定住，返回一个新的函数（偏函数）。

例如：int()函数可以将传入的字符串按规定进制进行转换，实际上它的定义为def int(string, base=10)

今欲定义一个函数，它将base固定为8（这样调用函数时实际上只能传入一个参数，否则会报错），作用是将数字字符串转换为8进制数：

```
import functools as f
```

```
int8 = f.partial(int, base = 8)
```

```
print(int8('12345', 2)) # base已经被固定住，不能再传入参数，会报错
```

```
print(int8('12345')) # 将12345转换为8进制数
```

eval函数十分强大，官方定义为：返回传入字符串的表达式的结果。就是说：将字符串当成有效的表达式来求值并返回计算结果。

1、计算一个算式字符串的结果

①`print(eval("1+2*7-4/2+9"))`#输出22.0

②`x=1`

`print(eval("x+2"))`#输出3

2、将输入为列表（元组、集合、字典）格式的字符串转化为列表（元组、集合、字典）来返回

```
x = eval(input())  
print(x)
```

`[1,2,3,4,5]`

`[1, 2, 3, 4, 5]`

进程已结束,退出代码0

`(1,2,3,4,5)`

`(1, 2, 3, 4, 5)`

进程已结束,退出代码0

`{1,2,3,1,2,4,5}`

`{1, 2, 3, 4, 5}`

进程已结束,退出代码0

`{"Alice":19, "Bob":18, "Cindy":20}`

`{'Alice': 19, 'Bob': 18, 'Cindy': 20}`

进程已结束,退出代码0

练习:



厦门大学
XIAMEN UNIVERSITY

1. 完成示例阶乘训练;
2. 查找资料, 了解 turtle 库的基本应用
3. 完成数码管绘制练习
4. 完成科赫曲线绘制
5. 结合所学知识, 尝试画一个五角星?
- 6*. 给你一个整数 x , 如果 x 是一个回文整数, 返回 `true`; 否则, 返回 `false`。回文数是指正序（从左向右）和倒序（从右向左）读都是一样的整数;